

**INSTITUTO
FEDERAL**
Santa Catarina

Funções em TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Funções

declarando

■ funções nomeadas

```
function saudacao(nome: string): string {  
    return 'olá ' + nome  
}
```

■ expressão de função

```
let saudacao = function (nome: string): string {  
    return 'olá ' + nome  
}
```

■ função seta

```
let saudacao = (nome: string): string => {  
    return 'olá ' + nome  
}
```

```
let saudacao = (nome: string): string =>  
    'olá ' + nome
```

Parâmetros

opcionais

- marcados com o símbolo ?
- devem vir depois dos obrigatórios

exemplos

```
function log(message: string, userId?: string) {  
    let time = new Date().toLocaleTimeString();  
    console.log(time, message, userId || "Não conectado");  
}  
  
log("Página carregada");  
// Logs "12:38:31 PM Página carregada Não conectado"  
  
log("Usuário conectado em", "da763be");  
// Logs "12:38:31 PM Usuário conectado em da763be"
```



Parâmetros

opcionais

- marcados com o símbolo ?
- devem vir depois dos obrigatórios
- podem ter valores padrão
 - neste caso, sem ?

exemplos

```
function log(message: string, userId = "Não conectado") {  
  let time = new Date().toLocaleTimeString();  
  console.log(time, message, userId || "Não conectado");  
}
```

```
log("Página carregada");  
// Logs "12:38:31 PM Página carregada Não conectado"
```

```
log("Usuário conectado em", "da763be");  
// Logs "12:38:31 PM Usuário conectado em da763be"
```



Parâmetros

parâmetros rest

- permitem qualquer número de parâmetros (*funções variádicas*)
- polimorfismo de funções

exemplos

```
function calcularArea(...args: number[]): number {  
  
    if (args.length === 1) {  
        return Math.PI * args[0] ** 2;  
    } else if (args.length === 2) {  
        return args[0] * args[1];  
    } else {  
        throw new Error("Número inválido de argumentos");  
    }  
}  
  
console.log(calcularArea(5));  
console.log(calcularArea(4, 6));  
  
function soma(...numbers: number[]): number {  
    return numbers.reduce((total, n) => total + n, 0);  
}  
  
soma(1, 2, 3);
```



Funções

definindo o tipo

- usar type
- utilizar apenas código de tipo, não utilizar valores
- criar assinatura do tipo resumida
- criar assinatura de tipo completa

exemplos

```
type Operacao = (a: number, b: number) => number;  
  
const soma: Operacao = (a, b) => a + b;  
const subtracao: Operacao = (a, b) => a - b;  
const multiplicacao: Operacao = (a, b) => a * b;  
const divisao: Operacao = (a, b) => a / b;  
  
type Operacao = {  
  (a: number, b: number): number  
};
```



Funções

sobrecarga

- permite definir diferentes assinaturas para a mesma função
- composta de
 - uma ou mais assinaturas da função sem corpo
 - uma única implementação compatível com todas as assinaturas

exemplos

```
type Operacao = {  
  (a: number, b: number): number  
  (a: string, b: string): string  
};  
  
const combinar: Operacao = (a, b): any => {  
  if (typeof a === "string" && typeof b === "string") {  
    return a.concat(b);  
  }  
  if (typeof a === "number" && typeof b === "number") {  
    return a + b;  
  }  
};
```



Funções

sobrecarga

- permite definir diferentes assinaturas para a mesma função
- composta de
 - uma ou mais assinaturas da função sem corpo
 - uma única implementação compatível com todas as assinaturas

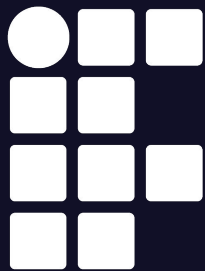
exemplos

```
function formatar(data: Date): string;
function formatar(texto: string, maiuscula: boolean):
string;

function formatar(a: Date | string, b?: boolean): string
{
    if (a instanceof Date) {
        return a.toISOString();
    }
    if (typeof a === "string") {
        return b ? a.toUpperCase() : a.toLowerCase();
    }
    return "";
}

console.log(formatar(new Date())); // ex:
2025-05-05T18:23:00.000Z
console.log(formatar("Olá Mundo", true)); // OLÁ MUNDO
console.log(formatar("Olá Mundo", false)); // olá mundo
```





**INSTITUTO
FEDERAL**
Santa Catarina

Funções em TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas